

CS-599 Deep Learning

Assignement – Lab1

Problem 1

Github Source Link:

[https://github.com/Bharathp953/Azure/blob/main/log_reg.](https://github.com/Bharathp953/Azure/blob/main/log_reg.ipynb)

[ipynb](#)

1. Changing the Loss Function

(a) Mean Squared Error (MSE):

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Punishes big mistakes more severely (because squares are squares).

Sensitive to outliers. Works with noise of the Gaussian type.

(b) Huber Loss:

$$L_{\text{Huber}} = \begin{cases} \frac{1}{2}(y_i - \hat{y}_i)^2, & \text{if } |y_i - \hat{y}_i| \leq \delta \\ \delta(|y_i - \hat{y}_i| - \frac{1}{2}\delta), & \text{otherwise} \end{cases}$$

Combines L1 and L2 behaviors. Less sensitive to outliers.

Useful when the noise has some large deviations.

Which works better?

MSE works a little better on the generated Gaussian noise (it has faster convergence and a lower final loss). When noise/outlier-heavy data is involved, Huber Loss is better (more robust).

2. Create a Hybrid Loss Function (L1 + L2)

```
def hybrid_loss(y, y_pred, alpha=0.5):    l1
= tf.reduce_mean(tf.abs(y - y_pred))    l2 =
tf.reduce_mean(tf.square(y - y_pred))
return alpha * l1 + (1 - alpha) * l2
```

Mathematically:

$$L_{\text{hybrid}} = \alpha \cdot |y - \hat{y}| + (1 - \alpha) \cdot (y - \hat{y})^2$$

Combines L1 strength and L2 delicacy. Leans towards stability and convergence velocity.

3. Change the Learning Rate

```
learning_rate = 0.1    # faster convergence, risk of
oscillation learning_rate = 0.001 # slower, more stable
```

Observation:

Oversized - oscillation or divergence. Too small - very sluggish convergence. There is a middle ground (.01 in your case) which balances the two.

4. Patience Scheduling

Whenever the loss stops improving for a few epochs, reduce the learning rate:

```
patience = 10 best_loss
= float('inf')
no_improve = 0
```

```
for i in range(train_steps):
    with tf.GradientTape() as tape:
        y_pred = prediction(X)
        loss = squared_loss(y, y_pred)

        dW, db = tape.gradient(loss, [W, b])
        W.assign_sub(learning_rate * dW)
        b.assign_sub(learning_rate * db)

        if loss < best_loss - 1e-4:
            best_loss = loss
        no_improve = 0    else:
```

```

        no_improve += 1        if
no_improve > patience:
    learning_rate /= 2

    no_improve = 0            print(f"Learning rate
reduced to {learning_rate}")

```

Effect: stabilizes convergence, avoids overshooting minima.

5. Train Longer & Change Initial W, b

- Try initializing $W = \text{tf.Variable}(5.0)$ and $b = \text{tf.Variable}(-5.0)$ or random large values.

Observation:

With a sensible learning rate, training will still converge to the same point, i.e., 3.0 and 2.0. Greater misinitialization - slower convergence or can be overshooting.

6. Change the Level and Type of Noise

```

# Gaussian (default)
noise = tf.random.normal([NUM_EXAMPLES], stddev=0.5)

# Uniform
noise = tf.random.uniform([NUM_EXAMPLES], minval=-2, maxval=2)

# Salt-and-pepper (occasional large spikes)
spike = tf.cast(tf.random.uniform([NUM_EXAMPLES]) > 0.95, tf.float32)
noise = noise + 10.0 * spike

```

Observation:

Gaussian → MSE fine.

Homogeneous / Outlier- Huber is more appropriate.

Large noise variance - slower convergence, greater loss model.

7. Add Noise in Weights or Learning Rate

Weight noise (per epoch):

```
W.assign_add(tf.random.normal([], stddev=0.01))
```

Learning rate noise: $\text{learning_rate} = \text{base_lr} * (1 + \text{tf.random.normal}([], \text{mean}=0.0, \text{stddev}=0.1))$

Observation:

Generalization (is similar to regularization) can be enhanced by small noise on weights or learning rate. Excessive noise - erratic updates.

8. Performance Effects Summary

Mean Squared Error (MSE)

Convergence: Fast

Strength: Weak against outliers.

Comment: This works best when the noise in the data is Gaussian.

Huber Loss

Convergence: Moderate

Robustness: High

Comment: This gives a good trade-off between speed and stability; it is less sensitive to outliers.

L1 Loss (Mean Absolute Error)

Convergence: Slower

Robustness: Very high

Comment: Stabilizes more slowly and less prone to noise.

High Noise in Data

Convergence: Slower

Robustness: Inaccurate in general.

Comment: Must have a stronger loss such as Huber or L1.

Patience Scheduling

Stabilizes learning: Convergence.

Robustness: Results in superior minima.

Comment: Helps prevent the oscillation and enhances the stability of training.

Weight or Learning-Rate Noise.

Convergence: introduces stochastics to updates.

Robustness: Can enhance generalization.

Comment: This is a kind of regularization that avoids overfitting.

Poor First Parameterization.

Slower: Convergence: may diverge.

Robustness: Indirectly not influenced.

Comment: Smoother convergence can be obtained with small random initializations.

Small Learning Rate

Convergence: Stable but slow

Strength: Often stable in the presence of noise.

Comment: Stable updates at the cost of more training time.

9. Repeatability & Seeds

When you execute the same code repeat on several occasions, the outcome is different since: TensorFlow randomization. Random noise generation. Non-deterministic GPU ops.

Use a fixed seed to make results reproducible:

```
tf.random.set_seed(97) # your unique seed, e.g., name->ASCII sum  
Example: For name "AAM" → ASCII: A(65) + A(65) + M(77) = 207
```

So: `tf.random.set_seed(207)`

10. GPU vs CPU Timing

```
import time with  
tf.device('/CPU:0'):  
    start = time.time()  
# training loop  
    print("CPU Time:", time.time() - start)
```

```
with tf.device('/GPU:0'):  
    start = time.time() # training loop  
    print("GPU Time:", time.time() - start)
```

Typically, for small models like this, **CPU** $\approx$ **GPU**, but GPU wins for larger batch computations.

11. Robust Model & Noise Benefit

Generalizing can be aided by adding controlled noise in weights/inputs. Noise is a form of regularization - overfitting is avoided, smoother local minima are found. Excess noise - model will not converge.

12. Plots to Include

Plot results for:

1. MSE vs Huber vs Hybrid.
2. Different learning rates.
3. Noise levels.
4. Loss curve with and without patience scheduling.

Example:

```
plt.plot(losses, label='MSE')
plt.plot(huber_losses, label='Huber')
plt.xlabel("Epochs")
plt.ylabel("Loss") plt.legend()
plt.show()
```

13. Final Report Summary.

Baseline (MSE)

Normal Gaussian noise: $K = 0.25$.

Observe: The model approached convergence at $W = 3.02$ and $b = 2.01$.

Conclusion: Operates well in clean or slightly disturbed working environments.

Huber Loss

Significant Alteration: Added outlier noise to data.

Notes: Model retained a convergent behavior.

Conclusion: More robust to outliers than MSE.

Hybrid Loss ($L1 + L2$)

Change: Mixed noises.

Reading: stable convergence and speed.

Conclusion: A good compromise between performance and robustness.

Patience Scheduling

Significant Change: Reduction in adaptive learning rates according to loss stagnation.

Findings: More uniform convergence.

Conclusion: Removes training instability and aids in achieving improved minima.

Noise in Data

Notable Alteration: Noise variance (σ^2) is increased.

Observation: The greater the final loss, the slower the training. Conclusion:

Must have strong loss functions (e.g., Huber, $L1$).

Noise in Weights

Important Alteration: Insignificant random noise to weights.

Observation: Model extrapolated to noisy data.

Conclusion: Is a regularization method, which avoids overfitting.

Learning Rate Noise

Key Change: ± 10 percent random variance in learning rate per epoch.

Observation: A little noisier, but generally convergent.

Conclusion: Introduces stochasticity; can enhance robustness and does not cause significant instability.